mind that serverless is an emerging approach. There are many challenges to be dealt with when developing serverless solutions. Let us discuss key challenges in the context of MovieBot example discussed earlier:

Debugging. Unlike typical application development, there is no concept of a local environment for serverless functions. Even fundamental debugging operations like stepping-through, breakpoints, step-over and watch points are not available with serverless functions. As of now, users need to rely on extensive logging and instrumentation for debugging.

When MovieBot provides an inconsistent response or does not understand the user's intent, debugging the MovieBot code (which is running remotely) requires users to log numerous details like NLP scores, dialogue responses and query results of the movie ticket database. Then they have to manually analyse and do detective work to find out what could have gone wrong.

State management. Although serverless is inherently stateless, real-world applications invariably have to deal with state. Orchestrating a set of serverless functions becomes a significant challenge when a common context has to be passed between them.

Any chatbot conversation represents a dialogue. It is important for the program to understand the entire conversation. For example, for the query "Is Dunkirk playing in Urvashi Theatre in Bangalore tonight?," if the answer from MovieBot is 'yes,' the next query from the user could be "Are two tickets available?" If MovieBot confirms, the user could say "Okay, book it."

For this transaction to work, MovieBot should remember the entire dialogue, which includes name of the movie, theatre location, city and number of tickets to book. This entire dialogue represents a sequence of stateless function calls. However, users need to persist this state for the final transaction to be successful. Maintaining this state external to functions is a tedious task.

Vendor lock-in. Although isolated functions are to be executed independently, users are in practice tied to the software development kit (SDK) and services provided by the serverless technology platform. This could result in vendor lock-in because it is difficult to migrate to other equivalent platforms.

Assume that you implement the MovieBot in AWS Lambda platform using Python. Though the core logic of the bot is written as Lambda functions, you need to use other related services from AWS platform—such as AWS Lex (for NLP), AWS API gateway and DynamoDB (for data persistence)—for the chatbot to work. Further, the bot code may need to make use of AWS SDK to consume services (such as S3 or DynamoDB), and that is written using boto3.

In other words, for the bot to be a reality, it needs to consume many services from the AWS platform rather than just the Lambda function code written in plain Python. This results in vendor lock-in because it is harder to migrate the bot to other platforms.

Other challenges. Each serverless function code typically would have third-party library dependencies. When deploying the serverless function, you need to deploy third-party dependency packages as well, which increases the deployment package size. Because containers are used underneath to execute serverless functions, the increased deployment size increases the latency to start-up and execute serverless functions. Further, maintaining all the dependent packages, versioning them, etc is a practical challenge as well.

Another challenge is serverless platforms' lack of support for widely used languages. For instance, as of May 2017, you could write functions in C#, Node.js (4.3 and 6.10), Python (2.7 and 3.6) and Java 8 on AWS Lambda. How about other languages like Go, PHP, Ruby, Groovy, Rust or any other language of your choice? Though there are ways available to write serverless functions in these languages and execute them, it is harder to do so. Since serverless technologies are maturing with support for a wider number of languages, this challenge will gradually disappear with time.

Serverless: A game-changer

Serverless changes the way you look at how applications are composed, written, deployed and scaled. If you want significant agility in creating highly scalable applications while ensuring cost-effectiveness, serverless is what you need. Businesses across the world are already providing highly compelling solutions using serverless technologies. Serverless has a wide range of applications from chatbots to real-time stream processing from IoT devices. So it is not a question of 'if,' but 'when' you will adopt serverless approach for your business. EFY

This article first appeared in September issue of EFY's Open Source For You magazine

Serverless technologies

AWS Lambda: https://aws.amazon.com/lambda/
Azure functions: https://functions.azure.com/
Google Cloud functions: https://cloud.google.com/functions/
Apache OpenWhisk: https://github.com/openwhisk
Serverless framework: https://github.com/serverless/serverless
Fission: https://github.com/fission/fission
Hyper.sh: https://github.com/hyperhq/
Funktion: https://funktion.fabric8.io/
Kubeless: http://kubeless.io/